

VEC Research Laboratory

Module 2

Multi-Vehicle marl Scheduling in Vehicular Edge Computing

Dr. Habte Lejebo

Department of Computer Science / Software Engineering
[Institution]

Research Areas: VEC · Multi-Agent RL · Fog Computing · Deep Reinforcement Learning

Prerequisites: Module 1 (single-vehicle VEC simulator)

Available at: github.com/habteL/VEC-DRL-Lab

Repository: github.com/habteL/VEC-DRL-Lab-Phase2

Contents

Preface	3
1 From Single-Agent to Multi-Agent VEC	4
1.1 Why Multi-Vehicle Matters	4
1.2 Parameter-Sharing MARL	4
1.3 The Contention Indicator	5
2 Multi-Vehicle System Configuration	6
2.1 Vehicle Configuration	6
2.2 Coverage Analysis	7
2.3 The Queue Cascade Effect	7
3 The Shared DQN Agent	8
3.1 State Vector Design	8
3.2 Reward Function Design	9
3.3 Randomised Vehicle Processing Order	9
4 Experimental Results	11
4.1 Baseline Comparison (C3a)	11
4.2 Load Regime Analysis (C3b)	11
4.3 Fairness Dynamics (C3c)	12
4.4 Scalability Analysis (C3d)	12
5 Implementation Guide	14
5.1 Repository Setup	14
5.2 Running the Experiments	14
5.3 Key Implementation Decisions	14
6 Laboratory Exercises	16
6.1 Understanding Questions	16
6.2 Implementation Exercises	17
6.3 Research Challenges	17
A Complete Sprint Summary	19
B File Structure	20

C Module 3 Preview

21

Preface

This module extends the single-vehicle VEC simulator from Module 1 into a Multi-Agent Reinforcement Learning (MARL) system where two or more heterogeneous vehicles compete for shared edge computing resources. It is designed for researchers who have completed Module 1 and want to study resource contention, fairness, load adaptation, and scalability in realistic VEC environments.

The module answers six research questions incrementally — each sprint adding one new concept while the previous is fully understood and verified.

Research Diary — The core lesson of this module

In multi-vehicle systems, the hardest problems are not algorithmic — they are environmental. Queue contention cascades, fairness under load, and scalability limits are all emergent properties of the shared infrastructure. The MARL agent's job is to navigate these dynamics, not eliminate them.

Chapter 1

From Single-Agent to Multi-Agent VEC

1.1 Why Multi-Vehicle Matters

In Module 1, one vehicle operated alone. It generated tasks, offloaded to servers, and the only competition was between its local CPU and the edge servers. In reality, dozens of vehicles share the same RSU infrastructure simultaneously.

Multi-vehicle VEC introduces three new challenges:

- **Resource contention** — two vehicles sending tasks to the same server simultaneously causes queue buildup that neither would experience alone.
- **Fairness** — the scheduler must ensure neither vehicle is systematically favoured over the other.
- **Scalability** — a policy trained on two vehicles should generalise to three, four, or more.

Research Note — Formal Multi-Vehicle VEC Problem

The multi-vehicle VEC problem extends the single-vehicle formulation. Given N vehicles $\mathcal{V} = \{v_1, \dots, v_N\}$ and M servers $\mathcal{R} = \{r_1, \dots, r_M\}$, find a joint scheduling policy π that maximises total task completion while maintaining fairness:

$$\max_{\pi} \sum_{i=1}^N C_i(\pi), \quad \text{s.t.} \quad J(\pi) \geq J_{\min} \quad (1.1)$$

where C_i is the completion rate of vehicle i and J is Jain's fairness index. The key difficulty: each vehicle's optimal action depends on what other vehicles are doing simultaneously — a Multi-Agent Decision Process.

1.2 Parameter-Sharing MARL

The key architectural decision in this module is **parameter sharing** — all vehicles use the same neural network weights. This means:

- One agent object is created and shared across all vehicles.
- Each vehicle observes its own local state and receives its own decisions.

- Learning happens from all vehicles' experiences simultaneously, doubling sample efficiency.
- The policy generalises across vehicles by design rather than by training.

The critical requirement: the state vector must include a **vehicle identity signal**, otherwise the agent cannot distinguish “I am vehicle 1” from “I am vehicle 2.”

1.3 The Contention Indicator

The most important new feature in the Phase 2 state vector is the contention indicator — a signal telling the agent how many other vehicles are currently sharing its server:

Listing 1.1: Contention indicator computation

```
1 # Number of OTHER vehicles on the same server right now
2 other_on_server = sum(
3     1 for other in vehicles
4     if other.vehicle_id != v.vehicle_id
5     and other.current_server_id == active_server.server_id
6 ) / 5.0 # normalise by max expected vehicles
```

This single feature allows the agent to detect the contention window and route tasks locally rather than adding to an already-congested server queue.

Chapter 2

Multi-Vehicle System Configuration

2.1 Vehicle Configuration

Phase 2 uses two heterogeneous vehicles with deliberately different speeds and starting positions to create realistic contention:

Listing 2.1: Two-vehicle setup

```
1 vehicles = [  
2     Vehicle(vehicle_id=1, x=0, y=0, speed=2, direction=0),  
3     Vehicle(vehicle_id=2, x=15, y=0, speed=1, direction=0),  
4 ]
```

Table 2.1: Vehicle configuration summary

Vehicle	Start x	Speed	Coverage Steps	Dead Zone Steps
V1	0	2 units/step	30 steps (0–29)	5 steps (30–34)
V2	15	1 unit/step	35 steps (0–34)	0 steps

Vehicle 2 starts at $x = 15$ (already inside Server 1 range) and moves slowly, ensuring it stays in coverage for the entire 35-step simulation. This creates a 10-step contention window (steps 10–20) where both vehicles simultaneously use Server 2.

Research Diary — Why $x=15$ specifically?

Starting V2 at $x = 0$ would make them identical — no research value. Starting at $x = 20$ would mean V2 enters Server 2 at step 0 and exits before V1 arrives — no contention window. The position $x = 15$ was chosen to create exactly 10 steps of simultaneous competition on Server 2, which is long enough to generate measurable cascade effects but short enough to keep the simulation tractable.

2.2 Coverage Analysis

Table 2.2: Per-step server coverage for the default two-vehicle configuration

Steps	V1 Server	V2 Server	Contention?
0–5	Server 1	Server 1	Yes — two-way
6–9	Server 1	Server 2	No
10–20	Server 2	Server 2	YES — peak contention
21–25	Server 3	Server 2	No
26–29	Server 3	Server 3	Yes — two-way
30–34	Dead zone	Server 3	No

2.3 The Queue Cascade Effect

The most important discovery in Sprint C1 was that contention does not just affect the contention server — it poisons downstream servers through handover migration.

What happens during and after the contention window:

1. **Steps 10–20:** Both vehicles flood Server 2. Queue grows rapidly (arrival rate $\approx 3 \times$ service rate).
2. **Step 20:** V1 hands over to Server 3. Handover migrates Server 2’s entire queue via backhaul.
3. **Steps 20–34:** Server 3 inherits a massive backlog plus new tasks from both vehicles.
4. **End of episode:** Server 3 has 36 tasks remaining.

Research Note — Research significance of the cascade

A researcher who only monitors server-level metrics during contention would miss the real damage — which appears steps later, on a different server, through the backhaul migration mechanism. This motivates contention-aware scheduling: the agent should route tasks locally during steps 10–20, not to prevent contention itself, but to prevent the downstream cascade.

Chapter 3

The Shared DQN Agent

3.1 State Vector Design

The Phase 2 state vector extends Phase 1’s 6 elements to 7, adding the contention indicator as the final feature:

Listing 3.1: Seven-element state vector

```
1 state = [  
2     edge_load / 50,                                # 1. total  
        server load  
3     len(active_server.transmission_queue) / 20,    # 2.  
        wireless queue  
4     task.cpu_cycles / 1000,                        # 3. task  
        size  
5     task.data_size / 1_000_000,                   # 4. data  
        volume  
6     len(v.local_queue) / 20,                      # 5. local  
        queue  
7     1 if v.local_task else 0,                     # 6. local  
        CPU busy  
8     other_on_server                               # 7.  
        contention (NEW)  
9 ]
```

where `edge_load` includes all queues to give a complete picture of server congestion:

Listing 3.2: Full edge load computation

```
1 edge_load = (  
2     len(active_server.task_queue) +  
3     len(active_server.transmission_queue) +  
4     len(active_server.backhaul_queue) +  
5     (1 if active_server.current_task else 0)  
6 )
```

Research Note — The Markov Property

For a DQN agent to learn correctly, the state must fully describe the environment. Adding `local_busy` (element 6) restored the Markov property for local execution decisions. Adding `other_on_server` (element 7) restored it for contention-aware decisions. Each addition made the state more Markovian, reducing the amount the agent had to infer from incomplete information.

3.2 Reward Function Design

The Phase 2 reward function has three components:

- **Immediate reward** — given when the scheduling decision is made.
- **Completion reward** (+1.0) — given when the task finishes.
- **Episode-end penalty** (−0.5 per unfinished task, per vehicle).

Listing 3.3: Symmetric contention-aware reward

```

1 contention_penalty = other_on_server * 0.3
2
3 # Local execution
4 local_pressure = (len(v.local_queue) + (1 if v.local_task
5     else 0)) / 5.0
6 immediate_reward = 0.2 - local_pressure - contention_penalty
7
8 # Edge offloading
9 edge_pressure = edge_load / 10.0
10 channel_cost = delay_steps / 3.0
11 immediate_reward = 0.2 - edge_pressure - channel_cost -
12     contention_penalty

```

Research Diary — Reward engineering lesson

The contention penalty must apply to *both* actions, not just offloading. If only offloading is penalised during contention, the agent always chooses local execution regardless of whether local queues are also saturated. The symmetric penalty forces the agent to compare true costs — congested local CPU vs. congested edge server — and choose the less overloaded resource.

3.3 Randomised Vehicle Processing Order

Each time step, vehicles are processed in a random order:

Listing 3.4: Randomised vehicle order per step

```

1 vehicle_order = random.sample(vehicles, len(vehicles))
2 for v in vehicle_order:
3     # process this vehicle's tasks

```

Without randomisation, V1 always gets first access to server capacity each step, systematically disadvantaging V2. Randomisation ensures the shared agent learns a general policy rather than one biased by fixed processing order.

Chapter 4

Experimental Results

4.1 Baseline Comparison (C3a)

The trained MARL agent was evaluated against two fixed-policy baselines over 100 episodes using a held-out random seed (123) not seen during training:

Table 4.1: Baseline comparison over 100 evaluation episodes

Policy	Completion Rate	Jain Index	vs. Offload
Always Offload	26.4%	0.978	baseline
Always Local	20.3%	0.990	−23.1%
MARL DQN Agent	46.0%	0.992	+74.2%

Research Note — Why always-offload underperforms

The always-offload baseline causes queue cascade congestion during the contention window. The MARL agent learned to route tasks locally during steps 10–20, preventing the downstream Server 3 backlog. This confirms the research hypothesis: contention-aware scheduling reduces cascade congestion.

4.2 Load Regime Analysis (C3b)

Three load regimes were evaluated by varying the task arrival rate:

Table 4.2: Performance across three load regimes

Regime	Arrival	λ (tasks/step)	ρ	Completion
Low	<code>randint(0,1)</code>	1.0	≈ 1.2	70.1%
Medium	<code>randint(0,2)</code>	2.0	≈ 2.4	45.5%
High	<code>randint(0,3)</code>	3.0	≈ 3.6	46.0%

Three findings stand out:

- Low load significantly outperforms medium and high (70.1% vs. 45%) because the system is closer to the stability boundary $\rho < 1$.
- Medium and high converge to *identical* performance — both are severely overloaded and capacity-limited.
- All three regimes show the valley of despair pattern: initial random performance, crash during early exploitation, gradual recovery.

Research Diary — Counterintuitive finding: learning is hardest under low load

Under high load, random decisions already perform poorly so there is little distance to fall. Under low load, random decisions appear deceptively good — the system has slack capacity and most random decisions succeed. When the agent starts exploiting its early inaccurate Q-values, it crashes from a high baseline. This non-monotonic learning difficulty with workload intensity is worth highlighting in a paper — it challenges the assumption that easier environments always produce faster learning.

4.3 Fairness Dynamics (C3c)

Per-vehicle completion rates and Jain index were tracked across all 1000 training episodes. The key finding: Jain fairness index stays near 1.0 *from episode 1* across all three regimes. It never collapses and recovers — it is high from the start.

Research Note — Structural fairness via parameter sharing

This is a stronger claim than “the agent learned fairness.” It means: parameter-sharing MARL provides *structural* fairness guarantees without explicit fairness objectives in the reward function. Both vehicles use the same network weights, so when their states are similar, their actions are similar, producing symmetric completion rates. This addresses a common MARL concern — that agents develop selfish policies — by showing the architecture prevents it by design.

4.4 Scalability Analysis (C3d)

A third vehicle (V3: $x = 5$, speed = 1.5) was added to test scalability. Two experiments were conducted:

Table 4.3: Scalability results — 2 vs. 3 vehicles

Configuration	Completion	Jain	Notes
2 vehicles (trained)	46.0%	0.992	Reference
3 vehicles (zero-shot)	38.1%	0.980	−17.2% degradation
3 vehicles (retrained)	37.5%	0.981	No improvement from retraining

The performance limit is **capacity-driven, not policy-driven**. System utilisation increases from $\rho = 3.6$ (two vehicles) to $\rho = 5.4$ (three vehicles) — both severely overloaded. The agent cannot overcome a structural capacity deficit regardless of how well it learns.

Fairness is maintained in both three-vehicle experiments ($\text{Jain} \approx 0.980$), confirming the structural fairness guarantee persists at scale.

Chapter 5

Implementation Guide

5.1 Repository Setup

Listing 5.1: Clone and install

```
1 git clone https://github.com/habteL/VEC-DRL-Lab-Phase2.git
2 cd VEC-DRL-Lab-Phase2
3 pip install -e .
```

5.2 Running the Experiments

Listing 5.2: Full experiment sequence

```
1 # Sprint C2: Train MARL agent (high load)
2 python experiments/simulation.py --arrival-max 3 --regime high
3
4 # Sprint C3a: Baseline comparison
5 python experiments/baseline_comparison_marl.py
6
7 # Sprint C3b: Load regime analysis
8 python experiments/simulation.py --arrival-max 1 --regime low
9 python experiments/simulation.py --arrival-max 2 --regime
   medium
10 python experiments/plot_regimes.py
11
12 # Sprint C3c: Fairness dynamics
13 python experiments/plot_fairness.py
14
15 # Sprint C3d: Scalability
16 python experiments/scalability_c3d.py
```

5.3 Key Implementation Decisions

Decision 1 — `vehicle.current_server_id` as attribute Each vehicle tracks its own server handover state as `self.current_server_id`. This is Separation of Concerns: server

tracking belongs to the vehicle, not to a central dictionary in `simulation.py`.

Decision 2 — `completed_tasks` as per-step event buffer `edge_server.completed_tasks` is cleared every step. It acts as an event buffer — the server reports which tasks finished this step, the simulator consumes them and clears. This enables per-vehicle completion counting without scanning historical records.

Decision 3 — outcome-based reward with deferred delivery The agent does not receive completion rewards immediately when a task is submitted. Rewards are stored in `pending_experiences` and delivered when the task actually completes. This aligns the learning signal with real outcomes rather than predicted ones.

Decision 4 — per-vehicle episode-end penalty Unfinished tasks are attributed to the vehicle that generated them. Each vehicle receives its own penalty proportional to its backlog, ensuring each agent is accountable for its own scheduling decisions only.

Chapter 6

Laboratory Exercises

6.1 Understanding Questions

Exercise 1: The Queue Cascade Effect

Describe the sequence of events from contention to cascade using the Server 2 $\rightarrow$ Server 3 handover scenario from Sprint C1. What metric would detect this effect and at which time step would it first appear?

Exercise 2: Structural Fairness via Parameter Sharing

Explain why two vehicles using the same neural network weights tend to receive similar completion rates, even without an explicit fairness objective in the reward function. Under what conditions might this structural fairness break down?

Exercise 3: The Markov Property

Without element 7 (`other_on_server`), the agent cannot distinguish a congested queue caused by its own past decisions from one caused by another vehicle. Why does this violate the Markov property? How does adding the contention indicator fix it?

Exercise 4: Capacity-Limited Scalability

The system utilisation ρ increases from 3.6 to 5.4 when a third vehicle is added. Use queuing theory to explain why no scheduling policy can achieve high completion rates when $\rho \gg 1$, regardless of how well it has been trained. What would need to change architecturally to achieve $\rho < 1$?

6.2 Implementation Exercises

Exercise 5: Add Vehicle Identity to State

Modify the state vector to include the vehicle ID as a normalised feature (`vehicle_id / num_vehicles`). Retrain the agent and compare performance with and without this feature. Does adding identity improve or hurt the learned policy?

Exercise 6: Fixed-Order Baseline

Remove the `random.sample()` randomisation and process vehicles in fixed order (V1 always before V2). Run 100 evaluation episodes and compare Jain index against the randomised version. By how much does fixed ordering reduce fairness?

Exercise 7: Contention Penalty Weight Ablation

Try penalty weights of 0.1, 0.3, 0.6, and 1.0 in `contention_penalty = other_on_server * weight`. Plot completion rate and Jain index for each. What weight achieves the best trade-off between throughput and fairness?

Exercise 8: Add a Fourth Server

Add `EdgeServer(server_id=4, capacity=300, x=70, y=0, coverage_radius=10)` to eliminate V1's dead zone. How does this change V1 vs. V2 fairness and overall completion rate? Is the improvement due to the agent or to the infrastructure change?

6.3 Research Challenges

Exercise 9: Stochastic Backhaul Bandwidth

Currently `backhaul_bandwidth` is fixed at 10 Mbps. Model cross-traffic congestion by making available bandwidth vary each step: `available = backhaul_bandwidth * random.uniform(0.2, 1.0)`. How does variable backhaul affect the cascade effect and the agent's learned policy?

Exercise 10: Deadline-Aware Scheduling

Tasks have a `deadline` attribute (currently unused). Modify the reward function to add a large penalty (-2.0) when a task completes after its deadline. Train the agent and compare deadline violation rates against always-offload and always-local. Does the agent learn to prioritise urgent tasks?

Exercise 11: Domain Randomisation for Scalability

Train the agent with randomly varying numbers of vehicles (1, 2, or 3) each episode. Evaluate on a fixed 3-vehicle configuration. Compare zero-shot scalability against an agent trained only on 2 vehicles. Does domain randomisation reduce the distribution-shift penalty observed in Sprint C3d?

Appendix A

Complete Sprint Summary

Table A.1: All Phase 2 sprints, goals, and outcomes

Sprint	Goal	Key Finding	Status
C1	Two-vehicle env.	Queue cascade effect discovered	✓
C2	Shared MARL agent	+74.2% over always-offload, Jain=0.992	✓
C3a	Baseline comparison	MARL dominates both baselines	✓
C3b	Load regimes	Graceful degradation; medium $\approx$ high	✓
C3c	Fairness dynamics	Structural fairness via param. sharing	✓
C3d	Scalability	Capacity-limited, not policy-limited	✓

Appendix B

File Structure

Listing B.1: Phase 2 repository layout

```
1 VEC-DRL-Lab-Phase2/
2 |-- src/vecsim/
3 |   |-- task.py           # unchanged from Phase 1
4 |   |-- vehicle.py        # + current_server_id, arrival_max
5 |   |   param
6 |   |-- edge_server.py    # + completed_tasks event buffer
7 |   |-- channel.py        # unchanged
8 |   |-- agent.py          # unchanged (state_size
9 |   |   parameterized)
10 |-- experiments/
11 |   |-- simulation.py      # MARL training (--arrival-max, --
12 |   |   regime)
13 |   |-- baseline_comparison_marl.py
14 |   |-- plot.py
15 |   |-- plot_regimes.py
16 |   |-- plot_fairness.py
17 |   |-- scalability_c3d.py
18 |-- results/
19 |   |-- trained_agent_marl_{low,medium,high}.npz
20 |   |-- trained_agent_3vehicles.npz
21 |   |-- episode_metrics_marl_{low,medium,high}.csv
22 |   |-- comparison_marl.png
23 |   |-- regime_comparison.png
24 |   |-- fairness_dynamics.png
25 |   |-- scalability_c3d.png
26 |-- docs/
27 |   |-- VEC_Lab_Manual_Phase2.pdf
28 |-- setup.py
29 |-- CITATION.md
30 |-- LICENSE
31 |-- README.md
```

Appendix C

Module 3 Preview

Module 3 will extend the simulator toward the full MAQS framework — Mitigation-Aware Quantum Scheduling — applying the same Agile development methodology to a quantum computing environment. Key additions:

- **QTask model** — qubit requirements, circuit depth, gate statistics.
- **QNode model** — IBM calibration data (T1/T2 coherence, gate error rates).
- **Fidelity estimation** — circuit transpilation-based quality score.
- **Error mitigation** — ZNE, PEC, and SV strategy selection.
- **PPO agent with action masking** — for infeasible node assignments.

The VEC simulator built in Modules 1 and 2 provides the conceptual foundation — tasks, queues, resources, agents — that maps directly to the quantum domain.